

SimpleSmartLoader

Overview

The Custom ELF Loader is a simple implementation of an ELF file loader in C. It loads 32-bit ELF executables manually without using the operating system's default loader. It handles page faults, simulates lazy loading of memory pages, and calculates total page faults, total pages allocated, and internal fragmentation in bytes and KB.

Header Files Used

stdio.h – For printing and input/output operations.
elf.h – For ELF header and program header structures.
string.h – For string functions like memset() and strerror().
fcntl.h – For file control operations such as open().
stdlib.h – For memory allocation and exit().
unistd.h – For file operations like read, lseek, and close.
sys/types.h – For system data types.
sys/mman.h – For memory mapping using mmap.
signal.h – For handling segmentation faults (SIGSEGV).
stdint.h – For fixed-width integer types like uintptr_t.
errno.h – For detailed error handling.

Functions

main()

Checks that the correct number of arguments are provided (./loader elf_file).
Sets up a signal handler for segmentation faults.
Calls ELF_loader_runner() to load and execute the ELF file.
After execution, calls Cleaner_loader() to free resources.
Calculates and prints total page faults, total pages allocated, and internal fragmentation in bytes and KB.

get_prot(uint32_t flags)

Converts ELF segment flags (PF_R, PF_W, PF_X) into mmap protection flags for read, write, and execute permissions.

ELF_loader_runner(char **argv)

Opens the ELF file and reads its ELF header.
Reads all program headers from the file.
Calculates the total memory required for PT_LOAD segments.
Stores the entry point address and calls run_program() to execute the ELF.

run_program(int (*_start)())

Runs the entry point function of the ELF file (_start).
Prints the return value of the executed ELF program.

seg_v_handle(int signal, siginfo_t *info, void *context)

This is the segmentation fault handler.
When a page fault occurs, it calls handle_page_fault() with the faulting address.

handle_page_fault(uintptr_t addr)

Handles lazy loading when a page fault happens.
Finds which segment contains the faulting address.
Allocates a new 4KB page using mmap.
Reads the required bytes from the ELF file into memory.
Fills the rest of the page (uninitialized memory) with zeros.
Increments page fault and page count counters.
If the address does not belong to any segment, it raises SIGSEGV again.

Cleaner_loader()

Frees allocated memory for ELF headers.
Closes the ELF file descriptor.
Cleans up all resources before program exit.

Link : <https://github.com/Swaraaj333/group-48>

Work Division

1)Prerak Tanwar 2024436

- a)run_program(int (*_start)())
- b)seg_v_handle(int signal, siginfo_t *info, void *context)
- c)handle_page_fault(uintptr_t addr)

2)Swaraaj Krishna 2024574

- a)main()
- b)get_prot(uint32_t flags)
- c)ELF_loader_runner(char **argv)

AI Policy

I have used the following AI generated code and implemented it after understanding it.

```
int prot = get_prot(phdr[i].p_flags);  
  
    //allocation of new page  
    void *map = mmap((void *)page_start, PG_SIZE, prot | PROT_WRITE,  
MAP_PRIVATE | MAP_ANONYMOUS | MAP_FIXED,-1, 0);
```

```
//making mem zero before file data  
if(fb_start > page_start) {  
    memset(map, 0, fb_start - page_start);  
}
```

```
        //making file data zero ,bss
        uintptr_t tail_begin = (fb_end > page_start) ? fb_end :
page_start;
        uintptr_t tail_end = (page_end <= segment_end) ? page_end :
segment_end;

        if(tail_end > tail_begin) {
            memset((char *)map + (tail_begin - page_start), 0, tail_end -
tail_begin);
        }
```